RFC-001

Pipeline de Análisis Inteligente de Artículos

Proyecto B — Cliente: MediaGroup LATAM — Modalidad 2.3 (Ollama local)

Estado	Borrador para revisión
Versión	1.0
Fecha	25 de octubre de 2026
Autores	Sofia Ximena Castro
Curso	Fundamentos de Arquitectura LLM · BSG Institute
Entregable	Documento de Diseño
Documentos relacionados	Diagrama C4 ·Backend · Frontend · Costos

1. Resumen

MediaGroup LATAM necesita procesar artículos periodísticos de forma automática y auditable: clasificarlos por categoría, extraer entidades, medir el sentimiento y generar un resumen SEO de menos de 160 caracteres. Este documento propone un pipeline de 7 etapas expuesto como API REST (FastAPI), con procesamiento asíncrono, inferencia 100% local sobre Ollama (llama3.2:3b), persistencia en SQLite y observabilidad por etapa (latencia, tokens y costo estimado).

La propuesta prioriza **simplicidad operativa**, **privacidad editorial** (ningún texto sale de la infraestructura) y **trazabilidad**. El documento también registra con honestidad las limitaciones de esta primera versión y las condiciones en las que convendría cambiar de decisión (sección 9).

2. Contexto y problema

Hoy el análisis editorial es manual: lento, inconsistente entre editores y difícil de auditar. El cliente pide un sistema que ingiera artículos desde su CMS mediante webhook, los analice con un LLM y entregue resultados estructurados a otro sistema. El análisis puede tardar segundos o minutos, pero el CMS no debe quedar bloqueado esperando.

2.1 Objetivos

- Responder al webhook de ingesta en < 200 ms, sin depender de la duración del análisis.
- Producir por artículo: categoría, entidades (persona, organización y/o lugar), sentimiento y resumen SEO < 160 caracteres.
- Registrar un span por etapa con latencia, tokens consumidos y costo estimado.
- Operar en Ubuntu 24.04 con una instalación reproducible mediante script.
- Exponer un frontend React con login, chat y dashboard básico.

2.2 No-objetivos de esta versión

- Alta disponibilidad, multi-instancia o escalado horizontal.
- Entrenamiento o ajuste fino (fine-tuning) del modelo.
- Evaluación de calidad con un conjunto etiquetado por el cliente (se propone como trabajo futuro).
- Fallback automático a un proveedor cloud (se discute en 6.3 y 9).

3. Requisitos

ID	Requisito	Métrica / criterio de aceptación
RF-1	Ingesta por webhook HTTP con schema JSON validado	400/422 ante payload inválido; 202 con id ante payload válido
RF-2	Limpieza de HTML a texto plano	Sin etiquetas, scripts ni entidades HTML en la salida
RF-3	Clasificación en catálogo cerrado de 9 categorías	Valor fuera de catálogo se normaliza a 'otros'
RF-4	Extracción de entidades y sentimiento	Entidades con tipo; sentimiento en {positivo, neutral, negativo}
RF-5	Resumen SEO	Longitud estrictamente < 160 caracteres (garantía dura por truncado)
RF-6	Exportación a DB y webhook de salida	Estado 'done' persistido; fallo de callback no invalida el análisis
RNF-1	Latencia de ingesta	< 200 ms (el trabajo pesado es en segundo plano)
RNF-2	Seguridad	JWT en todos los endpoints de negocio; rate limiting por usuario
RNF-3	Resiliencia	Timeout, 1 reintento y circuit breaker hacia Ollama
RNF-4	Observabilidad	Log JSON por evento y span por etapa
RNF-5	Reproducibilidad	Un script instala Ollama, el modelo y el backend en Ubuntu 24.04

4. Arquitectura propuesta

La arquitectura completa (contexto, contenedores y componentes) está en el entregable E1. En resumen: una SPA React consume una API FastAPI; la API valida y encola cada artículo, y un pipeline asíncrono dentro del mismo proceso llama a Ollama por HTTP, guarda en SQLite y notifica por callback.

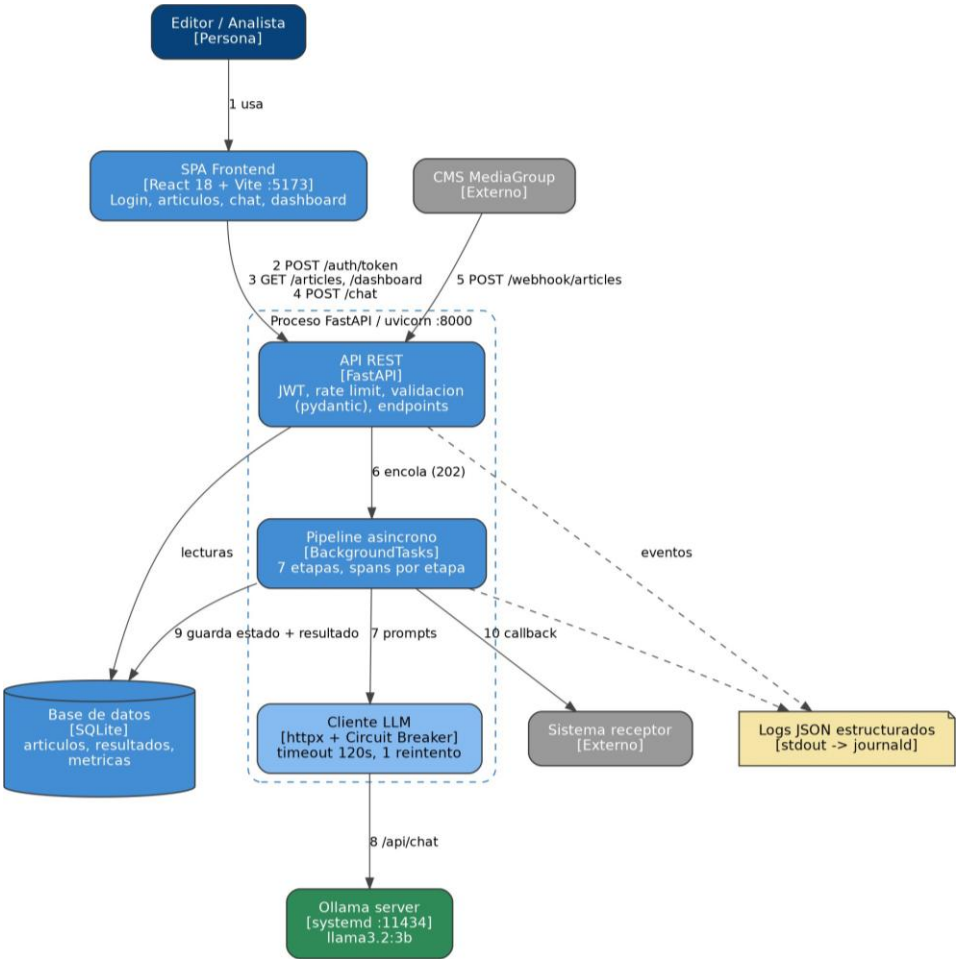


Figura 1. Contenedores y flujos de datos (C4 nivel 2).

5. Diseño detallado

5.1 Pipeline de 7 etapas

#	Etapa	Implementación	LLM	Falla si...
01	Ingest	POST /webhook/articles, responde 202 y encola con BackgroundTasks	No	JWT inválido (401), rate limit (429)
02	Validar	Pydantic (título 3-300, cuerpo hasta 100k) + cuerpo >= 50 caracteres	No	Payload inválido (422) o cuerpo corto
03	Limpiar	Quita script/style/etiquetas, desescapa entidades, normaliza espacios	No	Nunca (determinista)
04	Analizar	Una llamada JSON: categoría del catálogo	Sí	Ollama caído o breaker abierto
05	Enriquecer	Una llamada JSON: entidades y sentimiento	Sí	Ídem
06	Resumen SEO	Prompt de meta-description; reintento más corto; truncado duro a 157 + '...'	Sí	Ídem
07	Exportar	Persiste en SQLite y envía callback (timeout 10 s)	No	DB; el callback fallido solo se registra

El texto se recorta a 6.000 caracteres antes de cada llamada para respetar el contexto efectivo de un modelo de 3B y acotar la latencia.

5.2 Contrato de la API

Endpoint	Auth	Descripción
POST /auth/token	No	Credenciales -> JWT HS256 (1 h)
POST /webhook/articles	JWT	Ingesta; 202 {id, status: queued}
GET /articles, /articles/{id}	JWT	Listado y detalle con resultado y métricas
POST /chat	JWT	Asistente editorial libre sobre Ollama
GET /dashboard	JWT	Artículos, tokens, costo, latencia por etapa, estado del breaker
GET /health	No	Estado de API, Ollama y circuit breaker

5.3 Contrato de entrada y salida

```
POST /webhook/articles
{ "title": "Banco central sube tasas",
  "body": "<p>El Banco Central anunció...</p>",
  "url": "https://medio.com/nota", "callback_url": "https://cliente.com/hook" }

Resultado (GET /articles/{id}).result
{ "title": "...", "category": "economia",
  "sentiment": "negativo", "sentiment_score": -0.4,
  "entities": [{"text": "Banco Central", "type": "ORG"}],
  "seo_summary": "El Banco Central sube las tasas 25 puntos base...",
  "webhook_delivered": true }
```

5.4 Modelo de datos

Una tabla `articles(id, status, created_at, input, result, error, metrics)`. Los campos input, result y metrics son JSON serializado. Estados: queued, processing, done, failed. El campo metrics guarda la lista de spans por etapa, lo que permite reconstruir el costo y la latencia de cada artículo sin un sistema de métricas externo.

5.5 Seguridad

- JWT HS256 firmado con secreto desde variable de entorno; expiración de 20 minutos; comparación de credenciales en tiempo constante.
- Rate limiting por usuario con ventana deslizante de 60 s (30 solicitudes por defecto), respuesta 429 con Retry-After.
- CORS restringido al origen del frontend mediante variable de entorno.
- Ollama escucha solo en localhost: el modelo no se expone a la red.
- **Limitación:** un único usuario definido por variables de entorno y valores por defecto de desarrollo; para producción hay que exigir secretos reales y un almacén de usuarios.

5.6 Observabilidad

Cada etapa se envuelve en un span que registra etapa, latencia en ms, tokens, costo estimado y éxito o error, y lo emite como línea JSON con el id del artículo como trace_id. Además se registran eventos de negocio (login_ok, login_failed, article_ingested, rate_limited, circuit_opened, llm_error, webhook_failed, pipeline_done, pipeline_failed). El dashboard agrega esos mismos datos desde la base.

5.7 Resiliencia

- Timeout de 120 s por llamada y un reintento inmediato.
- Circuit breaker: abre tras 3 fallos consecutivos, rechaza llamadas 30 s (503) y pasa a half-open para sondear.
- Salida del modelo defensiva: modo JSON de Ollama, extracción de JSON por regex si hay texto extra, y normalización de valores fuera de catálogo.
- Un fallo de una etapa marca el artículo como failed con el motivo, sin afectar a otros artículos.

6. Decisiones y alternativas descartadas

6.1 Procesamiento: BackgroundTasks de FastAPI

Alternativa descartada	Cola externa (Celery + Redis) o cola gestionada (Cloud Tasks / Pub/Sub).
Justificación	El requisito crítico es responder en < 200 ms; BackgroundTasks lo cumple sin servicios adicionales.

Riesgo y mitigación	Las tareas en curso se pierden si el proceso se reinicia. Mitigación: el estado queued/processing queda en la base y puede reencolarse manualmente. Si el volumen o la criticidad crecen, migrar a una cola durable es la primera mejora.

6.2 Persistencia: SQLite

Alternativa descartada	PostgreSQL (Cloud SQL o local).
Justificación	Cero servicios que operar, un archivo, transacciones suficientes para el volumen esperado. El acceso está aislado en store.py, por lo que migrar es un cambio localizado.
Riesgo y mitigación	Escrituras concurrentes limitadas y un solo nodo. No apto para múltiples instancias.

6.3 Inferencia: Ollama local con llama3.2:3b

Alternativa descartada	Vertex AI (Gemini) como proveedor principal; fallback cloud automático.
Justificación	Es la modalidad 2.3 del curso, mantiene los textos dentro de la infraestructura y funciona sin conexión. El costo marginal por token es cercano a cero, aunque el costo fijo de la máquina no lo es (ver sección 8).
Riesgo y mitigación	Menor calidad que un modelo cloud grande y latencia alta en CPU. El enunciado describe un fallback Vertex AI hacia Ollama; en esta modalidad la dirección natural es la inversa (cloud como respaldo), pero **no está implementado** en esta versión. Queda como mejora y se compara en la documentación del modelo.

6.4 Tres llamadas LLM acotadas

Alternativa descartada	Un solo prompt que devuelva todo el análisis.
Justificación	Un modelo de 3B es más fiable con tareas pequeñas y salidas JSON cortas; cada prompt se puede depurar, acotar y medir por separado. Si una etapa falla, el artículo queda en failed con el motivo exacto.
Riesgo y mitigación	Más llamadas implican más tokens de entrada (el artículo se envía tres veces) y más latencia. Optimización posible: reutilizar el prefijo con el caché de contexto de Ollama.

6.5 Salida estructurada defensiva

Alternativa descartada	Confiar en que el modelo siempre devuelve JSON válido.
Justificación	Los modelos pequeños producen JSON inválido o valores fuera del catálogo con frecuencia no despreciable.
Riesgo y mitigación	Normalizar a 'otros' o 'neutral' puede ocultar errores de clasificación; por eso la calidad debe medirse con un conjunto de prueba (sección 10).

6.6 Resiliencia con circuit breaker propio

Alternativa descartada	Reintentos ilimitados o librerías externas de resiliencia.
Justificación	Evita acumular peticiones contra un Ollama caído o sobrecargado y libera rápido al pipeline. La implementación es de ~20 líneas y fácil de auditar.
Riesgo y mitigación	Un modelo en arranque en frío puede disparar falsos positivos; umbral y reset son configurables.

6.7 Autenticación: JWT con rate limit en memoria

Alternativa descartada	OAuth 2.0 / API Gateway / Redis para el rate limit.
Justificación	Un servicio único no justifica un proveedor de identidad ni un gateway.
Riesgo y mitigación	El contador de rate limit no se comparte entre instancias y se pierde al reiniciar.

7. Rendimiento esperado

Estas cifras son **hipótesis de diseño**, no mediciones. Se validarán con el benchmark descrito en la documentación del modelo.

Etapa LLM	Tokens de entrada	Tokens de salida	Total
04 Analizar	1.360	15	1.375
05 Enriquecer	1.420	150	1.570
06 Resumen SEO	1.350	50	1.400
Total por artículo	**4.130**	**215**	**4.345**

Supuesto: artículo promedio de unas 800 palabras (~1.300 tokens en español) más el prompt de sistema; sin reintentos del resumen. Con generación del orden de decenas de tokens por segundo en CPU, el análisis por artículo podría ir de decenas de segundos a un par de minutos; con GPU, de pocos segundos. Como Ollama procesa de forma serial por defecto, la capacidad de una instancia se estima en decenas de artículos por hora en CPU. El tamaño real de lote que soporta el cliente debe confirmarse con el benchmark.

8. Costos proyectados

El costo marginal de inferencia local es casi cero, pero existe un costo fijo de la máquina que ejecuta Ollama. El costo del proyecto que duro 4 días se fija a continuación.

8.1 Escenarios

El costo de dicho proyecto es basado en el tiempo que se dedicó en la elaboración del backend, frontend y los documentos. El costo de electricidad de baso en el costo promedio por dia en la México según la CFE, el tiempo de setup se sacaron de un desarrollador junio sin mucha experiencia.

Concepto	Costo
Costo del Tokens	$0
Costo de compute por 4 días	$73
Electricidad por dia 6	$24
Tiempo de setup 10 hora x 200	$2,000
Total	**$2,097**

8.2 Lectura

- Con estos supuestos, **el costo no justifica por sí solo el modo local** a volúmenes bajos y medios; lo justifican la privacidad, la independencia de proveedor y el cumplimiento de la modalidad del curso.
- El local gana en costo cuando el hardware ya existe (costo hundido) o cuando el volumen es alto y está respaldado por capacidad suficiente.
- La calidad también cuenta: un modelo cloud mayor puede evitar correcciones manuales, algo que este modelo de costos no captura.

9. Riesgos y limitaciones conocidas

Riesgo	Prob.	Impacto	Mitigación
Calidad insuficiente de llama3.2:3b en NER y categorías	Alta	Medio	Medir con conjunto de prueba; escalar a un modelo mayor en Ollama o añadir fallback cloud
Latencia alta en CPU	Alta	Medio	Procesamiento asíncrono; GPU o cuantización; reutilizar prefijo de contexto
Pérdida de tareas al reiniciar	Media	Alto	Reencolado desde la base; cola durable en una versión posterior
Resumen SEO truncado de forma poco natural	Media	Bajo	Reintento más corto antes del truncado; revisión humana por muestreo
Credenciales y secreto JWT por defecto	Media	Alto	Rechazar el arranque en producción si no se configuran
Entidades alucinadas por el modelo	Media	Medio	Verificar que la entidad aparezca en el texto (mejora pendiente)
Prompt injection desde el contenido del artículo	Media	Medio	Salida acotada a JSON y catálogos cerrados; el LLM no ejecuta herramientas

10. Plan de validación

- **Funcional:** pruebas de los 7 estados (payload inválido, éxito, Ollama caído, callback caído, rate limit, token expirado).
- **Calidad:** 30-50 artículos etiquetados a mano; medir exactitud de categoría, acuerdo de sentimiento y cobertura de entidades.
- **Rendimiento:** latencia y tokens reales por etapa a partir de los spans; comparar con la sección 7.
- **Comparativa:** el mismo prompt en Ollama y en una API cloud (calidad, latencia, costo), documentada en la ficha del modelo.